

1. What is a Class and how do you create it?

Answer:

A class is a blueprint for creating objects.

It defines properties (variables) and behaviors (methods).

Creating a Class:

```
class Student:
```

```
    def greet(self):
```

```
        print("Hello Student")
```

Creating an Object:

```
s1 = Student()
```

```
s1.greet()
```

Real-Time Example:

```
class Car:
```

```
    def start(self):
```

```
        print("Car started")
```

```
c1 = Car()
```

```
c1.start()
```

2. What is **init**, **self**, and **super** keyword?

Answer:

init Method:

It is a constructor that runs automatically when an object is created.

self Keyword:

Represents the current object. It is used to access instance variables.

Example:

```
class Person:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def display(self):
```

```
        print(self.name)
```

```
p1 = Person("Vasu")
```

```
p1.display()
```

super Keyword:

Used to call parent class constructor or methods.

Example:

```
class Parent:
```

```
    def __init__(self):
```

```
        print("Parent Constructor")
```

```
class Child(Parent):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        print("Child Constructor")
```

```
c = Child()
```

3. Explain Inheritance and its Types.

Answer:

Inheritance allows one class to acquire properties and methods of another class.

Single Inheritance:

class A:

```
def show(self):  
    print("Class A")
```

class B(A):

```
    pass
```

Multiple Inheritance:

class A:

```
    pass
```

class B:

```
    pass
```

class C(A, B):

```
    pass
```

Multi-Level Inheritance:

class A:

```
    pass
```

class B(A):

```
pass
```

```
class C(B):
```

```
    pass
```

Hierarchical Inheritance:

```
class A:
```

```
    pass
```

```
class B(A):
```

```
    pass
```

```
class C(A):
```

```
    pass
```

4. Explain Polymorphism (Overloading & Overriding).

Answer:

Method Overloading:

Python does not support traditional overloading, but it can be achieved using default arguments.

```
class Calculator:
```

```
    def add(self, a, b=0):
```

```
        return a + b
```

Method Overriding:

When a child class provides a different implementation of a parent method.

```
class Parent:
```

```
def show(self):  
    print("Parent method")
```

```
class Child(Parent):  
  
    def show(self):  
        print("Child method")
```

```
c = Child()  
c.show()
```

5. Explain Encapsulation and Data Abstraction.

Answer:

Encapsulation:

Binding data and methods together and restricting direct access.

Public: Accessible anywhere

Protected: `_variable` (convention)

Private: `__variable` (name mangling)

Example:

```
class Bank:
```

```
    def __init__(self, balance):  
        self.__balance = balance
```

```
    def get_balance(self):  
        return self.__balance
```

Abstraction:

Hiding internal implementation and showing only essential features.

Using ABC (Abstract Base Class):

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):
```

```
    @abstractmethod
```

```
    def area(self):
```

```
        pass
```

```
class Circle(Shape):
```

```
    def area(self):
```

```
        return "Calculating circle area"
```

Real-Time Example:

ATM machine – user only sees withdraw/deposit options, internal processing is hidden.

PART 2: QUIZ (MCQs)

1. Which method is called automatically when an object is created?

- A) start()
- B) init()
- C) **__init__()**
- D) constructor()

Correct Answer: C

2. Which keyword is used to call parent class constructor?

- A) parent
- B) this

- C) super
- D) base

Correct Answer: C

3. What does encapsulation achieve?

- A) Code duplication
- B) Data hiding Encapsulation **allows you to define controlled access to data stored inside objects of your class**
- C) Multiple inheritance
- D) Faster loops

Correct Answer: B